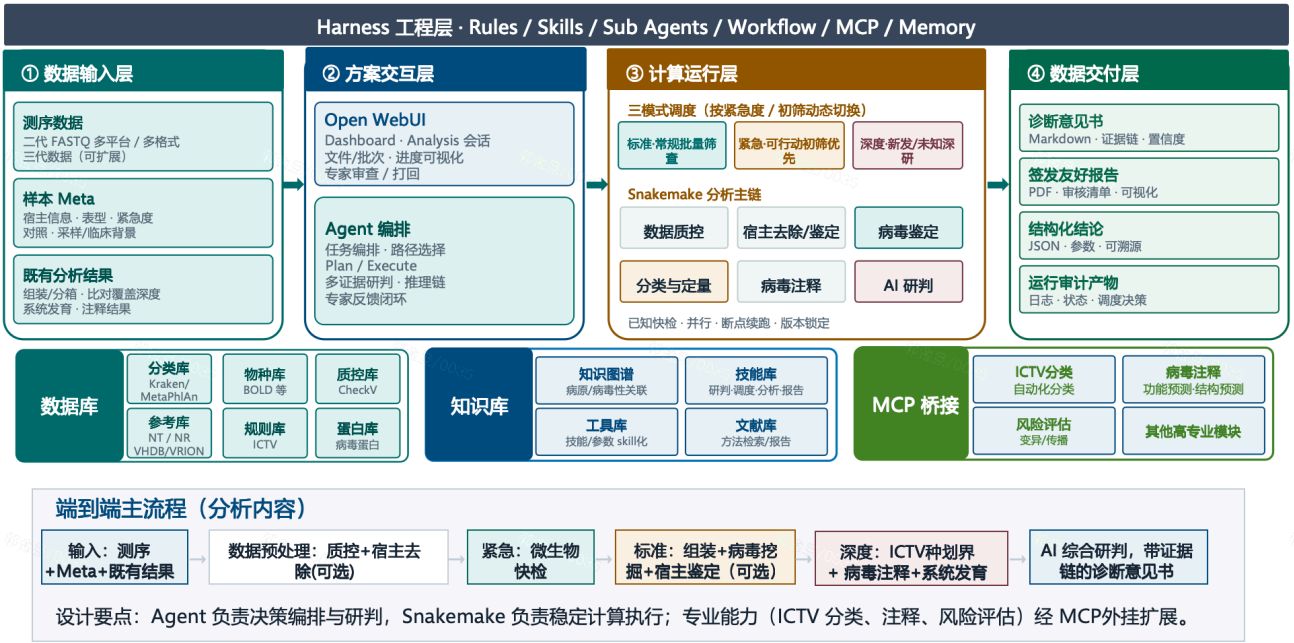
宏转录组未知病毒自动鉴定系统研究方案：AI驱动的Harness工程与智能决策框架

摘要

本研究旨在构建一套融合AI前瞻性能力的宏转录组未知病毒自动鉴定系统。基于现有Harness工程化架构（Rule、Skill、Sub Agent、Workflow、MCP）与Snakemake生产流水线，将三个核心AI能力——**专家级综合研判**、**应急智能调度与自优化**、**可解释推理驱动的人机互信和持续进化**——深度嵌入系统。系统从“固定产线”进化至“AI协诊专家”，实现从输入FASTQ到输出结构化鉴定报告的完全自动化，并具备动态资源调度、多源证据融合与持续学习能力。研究方案涵盖架构设计、模块实现、验证指标与实施路线，预期在20小时内完成分析，同时提供可审计、可进化、专家可监督的智能决策支持。

方案实施细节见： XvirMind Harness Engineering 规划与实施计划

v0616 修订：截至 2026-06-16，系统在 **cyclone**（`/data/bioagent`）已跑通「Open WebUI → BioAgent adapter → Claude Agent SDK → MiniMax-M3」聊天演示链路，并完成 Dashboard / System Status UI 骨架；Snakemake workflow1–6 已在 **172.24.195.66** 部署。本版正式采用 **双节点架构**（交互层 + 计算层），并将 Open WebUI fork 定为官方前端，后续工作重点由「UI 演示」转向 **Harness 内核回接、MCP 桥接、推理/调度模块落地**。



1. 研究背景与目标

1.1 问题域

需在60小时内完成宏转录组测序样本的病原鉴定，现有生信流程依赖人工操作，标准不一、判读依赖经验，存在以下核心挑战：

- **多指标矛盾**：高同源低覆盖、阈值边界模糊、EVE整合等生物学陷阱导致脚本误判。
- **应急响应僵化**：固定流程无法根据样本初筛结果、紧急程度动态调整分析深度与资源分配。
- **黑盒不可追溯**：脚本输出缺乏推理链，专家难以修正错误，系统无法从反馈中进化。

1.2 研究目标

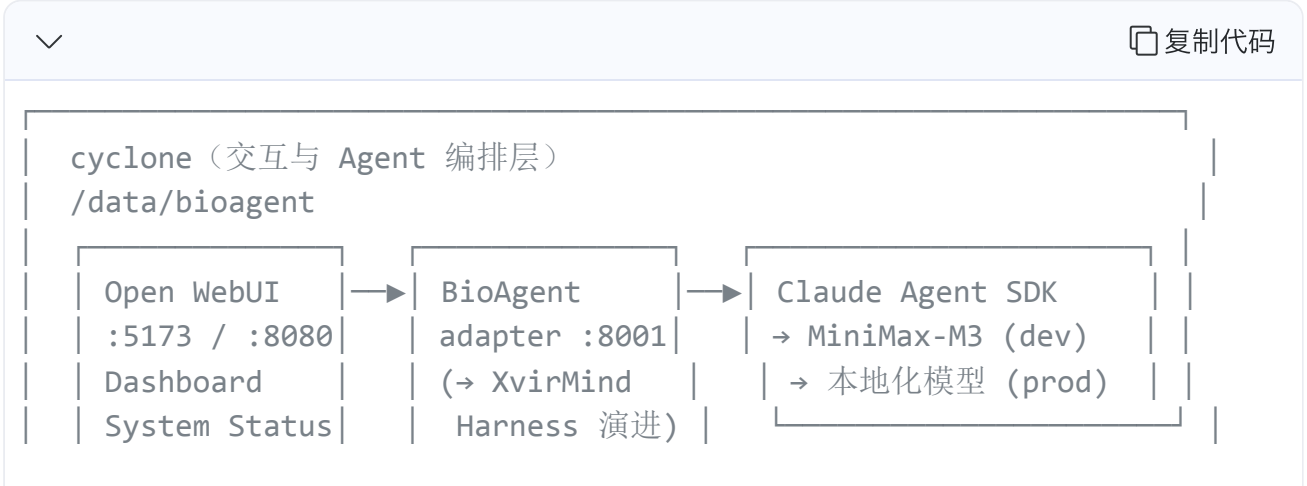
构建一套**AI驱动、可解释、自优化**的病毒鉴定系统，实现：

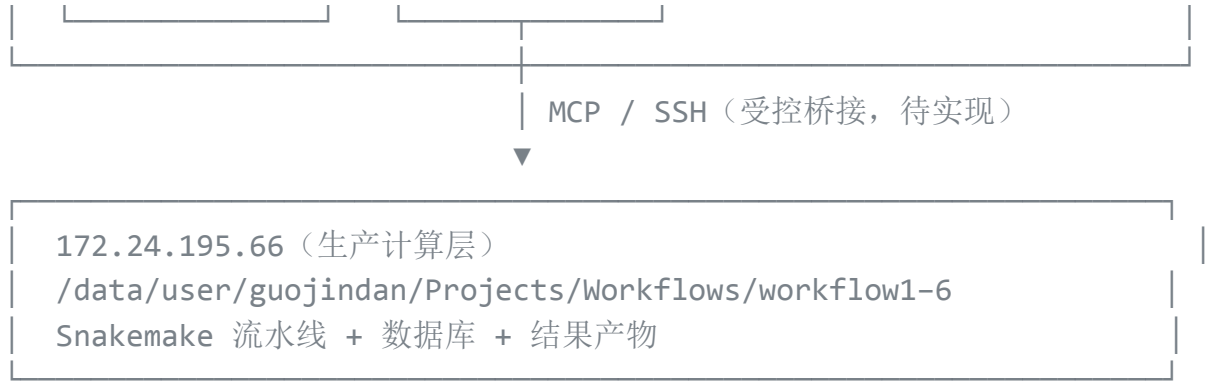
1. **专家级综合研判**：将碎片化生信数值（覆盖度、同源性、深度CV、系统发育位置）升维为符合病毒学家逻辑的结论。
2. **应急智能调度**：根据样本特征、剩余时间和任务优先级，动态切分分析路径（快速通道 vs. 完整链路）。
3. **可解释人机互信**：输出带完整推理步骤与证据引用的“诊断意见书”，支持专家审查、纠正与模型持续进化。

1.3 范围与约束

- **输入**：BGI PE150 FASTQ（gzip压缩），单样本或多批次。
- **输出**：结构化鉴定报告（Markdown + PDF + JSON），含病原体列表、证据链、置信度、可视化图表、审核清单。
- **非功能**：≤20小时，内存≤256GB，可断点续跑，多样本并行，版本锁定可重复。

1.4 部署拓扑（v0616 新增）





代码与 Harness 规范主仓：
/data/user/guojindan/Projects/XvirMind

---

2. 核心设计

2.1 专家级综合研判：从“查表投票”到“推理判断”

2.1.1 当前瓶颈

传统脚本（如Kraken2置信度阈值、BLAST同一性截断）本质是固定规则查表，无法处理真实数据的矛盾：

- **矛盾案例**：RdRp 91%同一性 + 覆盖率18% → 脚本直接判为“已知病毒”，实则可能为EVE或污染。
- **阈值边界问题**：89.7% 与 90% 无本质差异，但硬阈值导致截然不同的结论。

2.1.2 AI推理链设计

在Harness的 `review_agent` 和 `ai_report` 模块中嵌入**多证据融合推理器**，**核心组件**：

- **矛盾检测器**：对同一病毒候选，比较覆盖度、深度CV、一致性、系统发育支持度，识别“高同源低覆盖”、“高深度低一致性”等矛盾模 式。
- **阈值边界引擎**：不采用硬阈值（如90%），而是使用概率模型（高斯混合模型）评估同一性分布，对89.7%和90%同等对待，输出“接近阈值”标注。
- **生物学陷阱库**：嵌合Contig检测（CheckV结果）、内置EVE黑名单（人内源病毒元件）。
- **分级结论生成**：输出四层结论——确证、高置信、可疑、排除，并附证据权重。

2.1.3 与Harness集成

- **Rule层**：`rules/06_reasoning_rules.md`，规定AI必须显式处理矛盾与阈值边界。

- **Skill层**：`skills/report_skill.yaml` 增加“推理链验证”步骤，调用 `scripts/verify_reasoning.sh` 检查输出是否包含矛盾分析和不确定性标注。
- **Scripts**：新增 `scripts/verify_reasoning.sh`（验证推理链完整性），作为 `verify_all.sh` 的一个子脚本。

2.2 应急智能调度与自优化：从“固定产线”到“弹性指挥中心”

2.2.1 调度架构

在Harness的 `PMAgent` 和 `WorkflowEngine` 中引入**自适应路由**，基于实时数据动态选择分析路径。

决策依据：

- **紧急度**：用户指定（如“疫情爆发，5小时内需初步报告”）。
  - 样本输入 → 快速初筛（Kraken2 5% reads） → 调度器决策：
    - 路径A（紧急预警）：高置信已知病毒 → 跳过组装、系统发育 → 15分钟出报告
    - 路径B（标准分析）：未知或低置信 → 完整流程（M1-M8）
    - 路径C（深度探索）：孤儿序列 → 执行NR兜底 + 系统发育建树
- **样本特征**：Kraken2高置信命中数、总reads数、宿主去除后非宿主比例。
- **资源状态**：当前服务器负载、内存余量、已分配线程。
- **历史数据**：同类样本的平均耗时、成功路径概率（通过MCP调用 `system_resources` + `pipeline_history`）。

自优化机制：

- 每次分析后，将决策与结果（耗时、准确性、专家反馈）写入 `docs/dev-map/optimization_log.md`。
- 使用轻量级强化学习（Q-learning）更新调度策略，例如：当“跳过组装”导致漏检时，降低该路径的权重。

2.2.2 与Harness集成

- **Workflow层**：在 `workflow/definitions/flow_virus_analysis.yaml` 中增加 `adaptive_paths` 字段，定义不同路径的阶段序列。
- **MCP层**：`mcp/tools/pipeline_scheduler.py`，提供 `suggest_path(sample_metadata, urgency)` 接口。
- **Scripts**：`scripts/verify_scheduler.sh`，检查调度日志，确保紧急样本未被错误分入慢速路径。

2.3 可解释推理驱动的人机互信：从“黑盒结论”到“诊断意见书”

2.3.1 推理链显式化

在 `ai_report` 模块中，强制要求AI输出**思维链（Chain-of-Thought）**，并嵌入到报告中：

思维链 复制代码

```
## 证据与推理
### 候选病毒1：H5N1（甲型流感病毒）
- 支持证据：
  - Kraken2置信度：0.92（高）
  - 动态Bowtie2覆盖度：83%（>50%）
  - 深度CV：1.2（≤1.5）
  - Read一致性：0.94（≥0.9）
- 不确定点：
  - 未检出NS1基因（可能由于片段缺失或组装问题）
- 结论：**确证**，置信度：0.95
- 引用规则：FR-2（已知病毒检测），ICTV阈值文件第3.2节
```

2.3.2 专家交互接口

- **审核清单**：报告末尾嵌入结构化审核要点，如“请确认：是否考虑EVE污染？是否核对阴性对照？”。
- **打回机制**：专家可通过界面标记“反对”，并输入理由。系统自动捕获该反馈，存入 `docs/dev-map/pitfall_log.md`，并触发 `PMAgent` 重跑修正流程。
- **模型微调**：每月一次，使用积累的专家反馈（正例、负例）对Claude API的prompt进行微调（通过few-shot学习或检索增强）。

2.3.3 与Harness集成

- **Agent层**：`review_agent` 在审查输出时，检查推理链的完整性（是否每个结论都有至少两个支持/反对证据）。
- **Scripts**：`scripts/verify_reasoning.sh`，解析 Markdown 报告，检查推理链格式是否符合规范。
- **Skill层**：`skills/report_skill.yaml` 中增加“推理链验证”步骤。

---

3. 系统架构与模块集成

3.1 Harness全景扩展

整体架构

- Layer 1: SPEC 规格设计文档
- Layer 2: Rule 规矩层
- Layer 3: Skill 标准操作层
- Layer 4: Sub Agent 协作层

- Layer 5: Workflow 流程层
- Layer 6: Scripts 门禁层（验证脚本层）
- Layer 7: MCP 外接层
- Layer 8: dev-map 项目记忆层

新增组件	路径	功能
推理引擎	`core/review_agent.py`	在现有 `review_agent` 中增加合逻辑
自适应调度器	`core/adaptive_scheduler.py`	作为 `PMAgent` 的子模块
专家反馈收集器	`core/feedback_collector.py`	捕获专家审查标记
推理链验证脚本	`scripts/verify_reasoning.sh`	属于 Layer 6 验证脚本
调度日志验证脚本	`scripts/verify_scheduler.sh`	属于 Layer 6 验证脚本
优化日志	`docs/dev-map/optimization_log.md`	属于 Layer 8 项目记忆

3.2 数据流与决策循环

复制代码

```
FASTQ（195.66）
  → 样本初筛 Kraken2 5%（workflow4）
  → 调度器 suggest_path（cyclone PMAgent + MCP）
     ├─ 路径 A 紧急：快速报告（≤25 min）
     └─ 路径 B/C：标准/深度（workflow1→2→…）
  → 证据汇聚 evidence.json
  → review_agent 推理（矛盾检测、阈值、陷阱库）
  → ai_report 诊断意见书（Markdown + PDF + JSON）
  → Open WebUI Dashboard 展示 + 专家审查/打回
  → feedback_collector → optimization_log / pitfall_log
```

3.3 与现有Snakemake流水线的对接

通过MCP层实现Agent与生产级流水线的无缝交互，目前已经搭建的流程：

3.3.1 数据预处理流程

- 流程进展：已完成在172.24.195.66 （下同）部署，位置：/data/user/guojindan/Projects/Workflows/workflow1_upstream

3.3.2 病毒鉴定流程

- 流程进展：已完成在172.24.195.66 （下同）部署，位置：/data/user/guojindan/Projects/Workflows/workflow2_virusid
- 后续：数据库优化，NT库和NR库

3.3.3 宿主鉴定流程

- 流程进展：已完成在172.24.195.66 （下同）部署，位置：/data/user/guojindan/Projects/Workflows/workflow3_hostid
- 后续：需要更新BOLD库（组装 or read）

3.3.4 微生物分类流程

- 流程进展：已完成在172.24.195.66 （下同）部署，位置：/data/user/guojindan/Projects/Workflows/workflow4_kraken_bracken
- 后续：数据库的选择，结果输出方式；其他鉴定流程引入：MetaPhlAn，引入的话优先级或者综合判定 标准

3.3.5 病毒定量流程

- 流程进展：已完成在172.24.195.66 （下同）部署，位置：/data/user/guojindan/Projects/Workflows/workflow5_virus_quantification
- 后续：暂无

3.3.6 病毒注释流程

- 流程进展：输入鉴定后的病毒 FASTA 序列后，先用 CheckV 评估质量并过滤掉非病毒或完整度低于 50%的序列，然后根据病毒分类学信息（taxonomy）和序列长度决定结构注释策略——有 VIGOR模型（如 SARS-CoV-2、流感、Alpha冠状病毒）的病毒优先使用 **VIGOR**进行结构注释，其他病毒则用 Prodigal-meta 兜底，接着用 **DRAM-v** 对所有预测蛋白进行功能注释，最后做一致性校验并输出每条序列的 GenBank 文件、注释汇总表和一致性报告。位置：/data/user/guojindan/Projects/Workflows/workflow6_virus_annotation/
- 后续：
  - 流程还需要进一步测试
  - 基于生信流程还是搭建子agent待定

3.3.7 基因组补齐

- 待办

3.4 与其他外部agent的对接

这里我们采用的是“Agent 决策编排 + 生产流水线执行 + MCP 受控桥接”的调度方式，这种通过MCP层实现Agent与外部agent的无缝交互的方式把“专业能力”与“主流程编排”解耦，让 XvirMind 不需要在核心代码里反复重造 **ICTV 分类、病毒注释、风险评估**这些**高专业模块**，就能快速复用成熟能力并持续迭代；它同时提升了准确性（使用领域专用模型/规则）、扩展性（新增一个同类 Agent 只需加适配器）、可治理性（统一鉴权、审计、版本追踪、失败降级）和协作效率（算法团队与平台团队可并行演进）。

对疾控场景来说，这种模式本质上是把“业务灵活调度”和“生产稳定执行”同时保住：上层可按样本紧急度动态选路径，下层仍保持可控、可复现、可追责。

3.4.1 ICTV病毒分类Agent

📁ICTV病毒分类Agent

病毒分类实践面临三个核心痛点：

1. **标准分散且异质**：32个脊椎动物相关病毒科各自有不同的分类标准——有的用氨基酸相似度（如Picornaviridae P1蛋白>66%差异划分属），有的用遗传距离（如Paramyxoviridae L蛋白0.64-0.90 substitutions/site），有的无明确数值阈值（如Coronaviridae基于PPD聚类）。研究人员需要逐科查阅文献，学习成本极高。
2. **计算流程不统一**：不同科要求计算不同基因区域（RdRp、L蛋白、VP1、L1 ORF等）的不同指标（核酸%identity、氨基酸%identity、遗传距离等），使用不同的工具组合，缺乏统一入口。
3. **现有方法的局限性**：系统发育放置方法（如EPA-ng）虽能自动化分类，但本质上是"最近邻"方法，不等价于ICTV官方标准。**目前尚无能够严格遵循ICTV官方界定标准进行自动化分类的智能系统。**

核心目标：构建一个基于大语言模型（LLM）的智能代理系统，能够：

- **接受病毒序列（FASTA格式）作为输入**
- **自动识别所属病毒科**（通过BLAST/DIAMOND搜索）
- **查询该科的ICTV官方分类标准**（从结构化知识库中检索）
- **调用相应的生物信息学工具**（MAFFT、HMMER、BLAST等）执行标准要求的计算
- **比对计算结果与官方阈值**，给出分类判定（科/属/种）、置信度和证据链
- **支持新种判定**：当序列低于种级阈值时，标记为疑似新种

核心创新点：

用LLM作为推理引擎，将ICTV文档中的非结构化分类标准转化为结构化知识库，并驱动工具调用流程——不同于传统的固定流程Pipeline，Agent可根据不同科的标准动态选择计算方法和阈值。

3.4.2 病毒注释Agent （待办）

在3.3.6 部分搭建了基于生信方法的病毒注释流程，然而传统的生信方法依赖同源性比对，实际上病毒的**高度多样性**导致传统的基于同源性的方法失败，存在大量功能不明的“病毒暗物质”（40-90%的病毒基因缺乏功能注释），以及**计算复杂性**带来的瓶颈。因此这里拟采用**AI进行高灵敏度地初始扫描和发现，然后用传统的系统发育和功能注释方法进行验证和细化**，这结合了AI的发现能力与传统方法的可靠性。后续还可以在这个模块中，增加**结构预测**环节。

3.4.3 风险性评估Agent（待办）

传统的病毒风险评估方法高度依赖基于序列同源性的毒力因子、宿主受体motif等已知标记物比对，然而病毒的极度多样性、高频重组与快速变异使同源性方法常常失效，新发现的病毒中绝大多数致病风险与跨种潜能无法通过序列比对有效评估，形成了严重的“风险暗物质”（大量新型病毒的潜在威胁处于盲区），且多维度数据（基因组、流行病学、宿主生态、结构特征）融合带来的计算复杂性也成为瓶颈。因此这里拟采用AI进行高灵敏度的风险特征初始扫描与模式发现，捕捉潜在的毒力、免疫逃逸、跨种适应等多维风险信号，然后用传统的系统发育分析、正选择检测、毒力基因注释与宿主范围推断等方法进行验证和细化，实现AI的发现能力与传统方法的可靠性深度融合。后续还可以在这个模块中增加基于AI的蛋白质结构预测环节（如受体结合能力、融合蛋白构象评估），进一步提升对未知病毒致病性与传播风险的早期预判能力。

3.5 Agent模块技术实现：Claude Agent SDK

3.5.1 技术选型

本系统的 Agent 层（`ask_plan_agent` 、 `craft_agent`，`PMAgent`）统一采用 `claude-agent-sdk` **（Python）** 作为底层框架。该 SDK 以本地 Claude Code CLI 进程为执行引擎，通过 stdin/stdout JSON 协议与之通信，再由 CLI 进程调用 Anthropic API，形成完整的调用链：

Profile	场景	模型	端点
`dev-minimax`	cyclone 开发演示（**当前 bioagent**）	MiniMax-M3	api.minimaxi.com/anthropic
`prod-local`	195.66 合规/离线	待定本地化模型	内网推理服务
`subagent-cloud`	ICTV/注释/风险子 Agent	sonnet 或同级	经 SDK AgentDefinition

统一配置入口： `core/config.py` （`ClaudeAgentOptions` + 环境变量覆盖）。

核心优势：

- **多轮会话**：`ClaudeSDKClient` 天然支持跨轮次上下文保持，适合 PMAgent 的多步骤任务编排。
- **工具权限管控**：`can_use_tool` 回调可在每次工具调用前插入审批逻辑，满足"危险MCP操作需人类确认"的安全要求。
- **子Agent编排**：`AgentDefinition` 支持在主 Agent 内声明式定义子Agent（如 ICTV分类Agent），无需额外进程管理。
- **Hooks机制**：`PreToolUse` / `PostToolUse` 钩子可用于调度日志记录、推理链捕获等横切关注点。

3.5.2 骨架结构

复制代码

```
XvirMind/
├── core/
│   ├── adapter/              # [迁移目标] 自 bioagent/agent 迁入
│   │   ├── adapter.py
│   │   └── phases.py         # 改为 session-scoped
│   ├── agents/
│   │   ├── pm_agent.py
│   │   ├── review_agent.py
│   │   └── subagents.py
│   ├── config.py
│   └── tools/
│       ├── pipeline_tools.py
│       └── feedback_tools.py
├── mcp/
│   └── tools/
│       ├── snakemake_runner.py
│       ├── pipeline_scheduler.py
│       └── remote_pipeline_runner.py   # [新增] 跨节点
└── ...

/data/bioagent/                 # cyclone 运行态（部署目录，非第二套逻辑）
├── agent/                      # [过渡] 与 core/adapter 同步或软链
├── open-webui/                 # UI fork
├── conda/ / cache/ / data/
└── systemd-user/
```

3.5.3 关键骨架代码

统一配置（`config.py`）

复制代码

```
from claude_agent_sdk import ClaudeAgentOptions
from claude_agent_sdk.types import PermissionResultAllow,
PermissionResultDeny, ToolPermissionContext

DANGEROUS_TOOLS = {"delete_sample", "overwrite_database"}

async def _permission_handler(tool_name, input_data, context:
ToolPermissionContext):
    if tool_name in DANGEROUS_TOOLS:
        answer = input(f"[危险操作] {tool_name}({input_data}) 确认执行?
[y/N]: ")
        if answer.lower() != "y":
            return PermissionResultDeny(message="用户拒绝",
interrupt=False)
    return PermissionResultAllow(updated_input=input_data)

BASE_OPTIONS = ClaudeAgentOptions(
    system_prompt="你是XvirMind病毒鉴定系统的核心Agent，严格遵循Harness规则
层约束。",
    cwd="/data/user/guojindan/Projects/XvirMind",
    allowed_tools=["Read", "Bash", "mcp__pipeline__run_workflow"],
    can_use_tool=_permission_handler,
)
```

PMAgent（`pm_agent.py`）

复制代码

```
from claude_agent_sdk import ClaudeSDKClient, AgentDefinition
from .config import BASE_OPTIONS
from .subagents import ICTV_AGENT, ANNOTATION_AGENT
import dataclasses

options = dataclasses.replace(
    BASE_OPTIONS,
    agents={"ictv_classifier": ICTV_AGENT, "virus_annotator":
ANNOTATION_AGENT},
    allowed_tools=[*BASE_OPTIONS.allowed_tools, "Agent"],
)

async def run_pm_agent(sample_id: str, urgency: str):
    async with ClaudeSDKClient(options=options) as client:
        await client.query(f"处理样本 {sample_id}，紧急度={urgency}，执行
完整鉴定流程。")
        async for msg in client.receive_response():
```

子Agent定义（`subagents.py`）

复制代码

```
from claude_agent_sdk import AgentDefinition

ICTV_AGENT = AgentDefinition(
    description="根据ICTV官方标准对病毒序列进行分类鉴定",
    prompt="你是ICTV病毒分类专家，严格按照各科分类阈值输出分类结论和证据链。",
    tools=["Read", "Bash"],
    model="sonnet",
    maxTurns=20,
)

ANNOTATION_AGENT = AgentDefinition(
    description="对病毒序列进行功能注释，结合AI扫描与传统同源比对",
    prompt="你是病毒基因组注释专家，先用AI识别潜在功能域，再调用DRAM-v验
证。",
    tools=["Read", "Bash"],
    model="sonnet",
    maxTurns=15,
)
```

自定义MCP工具（`pipeline_tools.py`）

复制代码

```
from claude_agent_sdk import tool, create_sdk_mcp_server
from typing import Any

@tool("run_workflow", "触发指定Snakemake流水线", {"workflow_id": str,
"sample_id": str, "extra_args": str})
async def run_workflow(args: dict[str, Any]) -> dict[str, Any]:
    import subprocess
    cmd = f"snakemake -s workflows/{args['workflow_id']}.smk --config
sample={args['sample_id']} {args.get('extra_args', '')}"
    result = subprocess.run(cmd, shell=True, capture_output=True,
text=True)
    if result.returncode != 0:
        return {"content": [{"type": "text", "text": result.stderr}],
"is_error": True}
    return {"content": [{"type": "text", "text": result.stdout}]}

pipeline_server = create_sdk_mcp_server(name="pipeline", tools=
[run_workflow])
```

3.5.4 与Harness层的映射关系

Harness层	Claude Agent SDK 对应实现
Layer 4 Sub Agent	`AgentDefinition`（ICTV、注释、风险评估子Agent）
Layer 7 MCP 外接层	`@tool` + `create_sdk_mcp_server()` 桥接 Snakemake 流水线
安全门禁	`can_use_tool` 回调拦截危险操作
调度日志	`PostToolUse` Hook 写入 `optimization_log.md`
专家打回机制	`interrupt()` 中断当前任务，重新注入专家修正指令

---

4. 研究内容与关键技术

4.1 多证据融合推理模型

- **输入**：evidence.json（含Kraken2置信度、Diamond同一性、覆盖度coverage、CheckV结果、系统发育bootstrap，NR idt等）。
- **方法**：采用贝叶斯网络或轻量级图神经网络，对各证据进行概率加权，输出后验概率分布。
- **输出**：每个候选病毒的置信度（0-1）及矛盾标识（如“高同源低覆盖”）。
- **验证**：使用模拟数据集（含分节段病毒、阈值边界案例）评估F1-score。

4.2 自适应调度强化学习

- **状态空间**：样本特征（非宿主reads数、Kraken2高置信比例）、资源占用（CPU/内存）、剩余时间、紧急度。
- **动作空间**：选择不同路径（标准/紧急/深度）、调整线程数、跳过可选步骤。
- **奖励函数**：准确率（与专家判断一致）+ 耗时惩罚（超过20小时扣分）+ 资源利用率。
- **实现**：使用DQN（深度Q网络）离线训练，线上推理采用Q表匹配。每周一次模型更新。

4.3 可解释推理链生成

- **方法**：在本地化大模型的prompt中强制输出JSON格式的思维链，包含证据引用、矛盾识别、阈值决策理由。

- **检查**：推理链验证器解析JSON，检查证据是否与原始数据一致，是否存在逻辑跳跃。
- **反馈**：专家打回时，自动记录错误类型（如“误判覆盖度”），用于后续prompt优化。

4.4 人机互信迭代机制

- **反馈数据**：结构化存储专家审查记录（样本ID、AI结论、专家判断、修改理由、时间戳）。
- **模型更新**：每月基于反馈数据生成few-shot示例，更新system prompt；同时调整规则引擎的权重。
- **安全审计**：所有反馈数据不可修改，日志保存至少2年。

---

5. 实施计划与里程碑

阶段0：基线搭建（基本完成）

- 完成Claude.md/本地化模型、dev-map、pipelines registry、流程模板。
- 部署基础Harness架构（Rule、Skill、Sub Agent框架）。
- 功能模块设计和UI界面设计（优化中）
  - 项目展示页 Dashboard — 全局监控 + 时间演进 + 地理分布 + 历史回溯 + 智能问答
  - 操作页面- 文件上传 / 场景配置（标准/紧急） / 流程配置（多轮问答/可视化流程/结果预估） / 资源使用情况





阶段1：推理引擎与调度器

- 实现 `reasoning_engine.py` ，集成矛盾检测和阈值处理。
- 实现 `adaptive_scheduler.py` ，基于规则+轻量RL（**强化学习**）。
- 集成MCP调度工具。
- 验证紧急通道与标准通道的切换逻辑。

阶段2：可解释报告与专家反馈

- 扩展 `ai_report` 模块，输出结构化推理链。
- 实现 `feedback_collector.py` ，支持专家打回与标注。
- 实现推理链验证。
- 完成第一版“诊断意见书”模板。

阶段3：自优化闭环

- 收集至少100个模拟样本的调度与推理数据。
- 训练强化学习调度器，验证收敛性。
- 构建专家反馈数据集，进行prompt微调。
- 完成端到端集成测试（含完整流程、紧急场景、打回重跑）。

阶段4：生产验证与交付

- 真实样本（含已知阳性、阴性、EVE干扰）测试，统计准确率、召回率、平均耗时。
- 专家双盲对比实验（AI报告 vs. 人工分析）。
- 部署文档、运行手册、灾难恢复方案。
- 最终验收：20小时内完成，推理链完整，反馈可追溯。

---

6. 验证指标与评估方法

指标	目标值	测量方法
已知病毒检测准确率	≥95%	与专家标注的金标准对比
属内新种判定F1-score	≥0.85	使用模拟病毒新种数据集
假阳性过滤率	≥99%	阴性样本（无病毒）误报率≤1%
紧急通道耗时	≤25分钟（单样本）	在单线程模式下测量
标准通道耗时	≤16小时	真实样本测试（含并行）
推理链完整性	100%	门禁脚本检查通过率
专家接受率	≥90%	专家对AI结论的直接采纳比例（不修改）
调度决策优化收益	≥30%耗时降低	对比固定流程（相同资源）
反馈迭代后准确率提升	≥5%	对比未迭代的初始版本

---

7. 风险分析与应对

风险	概率	影响	缓解措施
大模型推理稳定性不足	中	高	利用标准数据训练，并设降级规则引擎
RL调度器初期表现差	中	中	先部署基于规则的调度器作为基线，RL逐步上线，设安全回滚
专家反馈数据质量低	低	中	强制专家填写结构化反馈表单，并设置校验规则

矛盾检测漏判	中	高	建立手工验证的“困难样本集”，定期回溯测试并更新检测算法
超时（>20小时）	低	高	多级超时控制：MCP层设置进程超时，PMAgent监控总耗时，超限自动降级
安全问题（MCP危险操作）	低	高	所有危险MCP工具（删除文件、修改数据库）需人类确认，日志审计

---

8. 预期成果与创新点

8.1 预期成果

1. **软件系统**：完整可运行的AI驱动病毒鉴定系统，含 UI界面、Harness架构、推理引擎、调度器、反馈循环。
2. **报告模板**：带推理链的“诊断意见书”Markdown模板及PDF导出。
3. **数据集**：包含100+模拟/真实样本的基准测试集，含标注、矛盾案例、专家反馈。
4. **文档**：设计文档、部署指南、运维手册、培训材料。

8.2 亮点

- **将多证据矛盾处理与阈值边界模糊性融入生信AI判读**，实现类专家推理。
- **提出面向应急任务的动态调度框架**，基于实时数据与RL优化，超越固定流程。
- **建立可解释推理链与专家反馈闭环**，使AI系统能够持续进化，且完全可审计。
- **基于Harness工程化方法**，将AI能力与Rule/Skill/Gate/MCP等工程约束有机结合，确保稳定可靠。
- 数据兼容性：不同测序平台二代测序数据通用

---

9. 后续展望

本系统设计为可扩展的病原鉴定平台，未来可：

- 扩展至细菌、真菌鉴定（只需替换证据规则和数据库）。
- 接入长读长测序，利用自适应调度器支持混合组装。
- 建设跨实验室联邦学习网络，积累多源专家反馈，提升泛化能力。
- 与公共卫生应急系统对接，实现实时预警与溯源。